

AZURE DATA ENGINEERING JOURNEY

Date Functions

13 date functions, two datasets — hands-on SQL practice notes from today's session.

SWIPE TO LEARN →

CLOUD SETUP · AZURE



Cloud-to-Desktop Workflow

02

Reconnected to the same Azure SQL Database via SSMS set up on Day 2 — this time uploading two fresh datasets to fresh datasets to practice SQL date functions in real time.

- ✓ Azure SQL Database reconnected via SSMS
- ✓ E-Commerce dataset uploaded and query-ready
- ✓ Superstore dataset uploaded and query-ready
- ✓ ORDERS table used as the base for date practice
- ✓ 13 date functions executed against live data



E-COMMERCE DATASET

Customer orders, shipping details, and regional sales sales across multiple order dates.



SUPERSTORE DATASET

Product-level sales, profit, and the [ORDER DATE] column used throughout this carousel.

TODAY'S TOOLKIT

06

 13 Date Functions, One Map

A quick map of every date function practiced today — swipe through to see each one run against the ORDERS table.

01 CURRENT MOMENT CURRENT_TIMESTAMP · GETDATE · GETUTCDATE · SYSDATETIME

02 DATE PARTS YEAR · MONTH · DAY

06 READ FORMAT DATENAME · DATEPART

04 SHIFT DATES DATEADD

05 MONTH BOUNDARIES DATEFROMPARTS · EOMONTH

06 MEASURE THE GAP DATEDIFF

07 REAL-WORLD QUERY Year-to-Date (YTD) filter

DATE FUNCTIONS · CURRENT DATE & TIME

04

Capturing Today's Date & Time

Four functions for grabbing the current moment — local server time, UTC time, and full-precision timestamps, straight from timestamps, straight from SQL Server.



CURRENT_TIMESTAMP

ANSI SQL standard property for today's date and time together — no parentheses needed, reference it directly in a SELECT.

```
-- CURRENT_TIMESTAMP gives date + time
SELECT CURRENT_TIMESTAMP
AS CURRENT_TIMESTAMP_
```



GETDATE()

SQL Server's native function for the current local date and time — the single most common 'right now' function in function in T-SQL.

```
-- GETDATE() gives today's date & time
SELECT GETDATE() AS DATE_
```



GETUTCDATE()

Returns the current UTC date and time instead of local server time — essential once your data spans more than one time zone.

```
-- GETUTCDATE() gives UTC timestamp
SELECT GETUTCDATE() AS UTC
```



SYSDATETIME()

Returns the current server date and time with higher fractional-second precision than GETDATE() — built for high-accuracy logs.

```
-- SYSDATETIME() gives the server time
SELECT SYSDATETIME() AS SERVER_TIME
```



Need IST instead of server time? SWITCHOFFSET(SYSDATETIMEOFFSET(), '+05:30') AS IST converts the server's timestamp straight to India Standard Time — the exact line used in this session's practice.

DATE FUNCTIONS · EXTRACTING PARTS

05

YEAR, MONTH & DAY

Three functions for pulling individual components out of a date column — the building blocks for grouping, filtering, and trend reports.



YEAR

Extracts just the year as an integer — integer — perfect for grouping yearly totals or filtering one specific year from ORDERS.

```
SELECT
  [ORDER DATE],
  YEAR([ORDER DATE])
  AS YEAR_
FROM ORDERS
```



MONTH

Extracts just the month as an integer integer (1–12) — the function behind every month-over-month trend chart.

```
SELECT
  [ORDER DATE],
  MONTH([ORDER DATE])
  AS MONTH_
FROM ORDERS
```



DAY

Extracts just the day-of-month as an integer (1–61) — useful for delivery patterns or billing-cycle cutoffs.

```
SELECT
  DAY([ORDER DATE])
  AS DAY_
FROM ORDERS
```

💡 DATENAME() and DATEPART() work the same way for every date part shown here — swipe ahead to see the difference between their string and integer output.

DATE FUNCTIONS · READING DATE PARTS

06

DATENAME vs DATEPART

Same job, different output — one hands back a readable string, the other a clean integer you can sort, filter, and do math on.

T DATENAME

Returns the date part as a readable string — e.g. the weekday name instead of a number. Great for labels in reports and reports and dashboards.

```
-- DATENAME returns the output as a string
SELECT [ORDER DATE], DATENAME(WEEKDAY,[ORDER DATE])
  AS WEEKDAY_ FROM ORDERS
SELECT [ORDER DATE], DATENAME(QUARTER,[ORDER DATE])
  AS Q_NO FROM ORDERS
```

DATEPART

Returns the date part as an integer — ideal for filtering, sorting, or feeding straight into calculations and WHERE clauses.

```
-- DATEPART returns the output as an integer
SELECT [ORDER DATE], DATEPART(WEEKDAY,[ORDER DATE])
  AS WEEKDAY_ FROM ORDERS
SELECT [ORDER DATE], DATEPART(QUARTER,[ORDER DATE])
  AS Q_NO FROM ORDERS
```

💡 Both functions accept the exact same date-part keywords — YEAR, MONTH, DAY, WEEKDAY, QUARTER — so once you know one, you know the other.

DATE FUNCTIONS · SHIFTING
DATES

▶▶ DATEADD

07

Adds or subtracts a chosen interval — year, month, or day — from a date. A negative number moves backward, a positive number moves forward, all in one function.

```
-- DATEADD(interval, number, date_column)
SELECT [ORDER DATE],DATEADD(YEAR,-1,[ORDER DATE]) AS PREVIOUS_YEAR FROM ORDERS
SELECT [ORDER DATE],DATEADD(YEAR,1,[ORDER DATE]) AS NEXT_YEAR FROM ORDERS
SELECT [ORDER DATE],DATEADD(MONTH,-1,[ORDER DATE]) AS PREVIOUS_MONTH FROM ORDERS
```

SYNTAX: DATEADD(datepart, number, date)

A negative number shifts backward and a positive number shifts forward — the same function drives both directions.

COMMON USE CASES

Previous vs next period comparisons

Rolling 6-month trend windows

Shifting due dates & SLA deadlines

DATE FUNCTIONS · MONTH
BOUNDARIES

08

Start of Month vs End of Month

Two ways to anchor a date to its month boundary — the first day for period-start reporting, the last day for month-end close.



DATEFROMPARTS · Start of Month

Builds a new date from separate year, month, and day values — pairing it with YEAR(), MONTH(), and a hardcoded 1 always returns the 1st of that month.

```
-- START OF MONTH
SELECT DATEFROMPARTS(YEAR([ORDER DATE]), MONTH([ORDER DATE]),1)
       AS START_OF_MONTH FROM ORDERS
```



EOMONTH · End of Month

Purpose-built to return the last calendar day of the month for any date — no manual day math, and it automatically adjusts automatically adjusts for leap years.

```
-- END OF MONTH
SELECT EOMONTH([ORDER DATE]) AS END_OF_MONTH_DATE FROM ORDERS
```

💡 EOMONTH() also accepts an optional second parameter to shift by whole months — EOMONTH(date, 1) jumps straight jumps straight to next month's end.

DATE FUNCTIONS · MEASURING
GAPS **DATEDIFF**

09

Calculates the difference between two dates in the interval you choose — year, quarter, month, week, or day — perfect for order-age or turnaround-time metrics.

```
-- DATEDIFF(datepart, start_date, end_date)
SELECT [ORDER DATE], DATEDIFF(YEAR,[ORDER DATE],GETDATE())
    AS YEAR_DIFF FROM ORDERS
SELECT [ORDER DATE], DATEDIFF(QUARTER,[ORDER DATE],GETDATE())
    AS QUARTER_DIFF FROM ORDERS
SELECT [ORDER DATE], DATEDIFF(MONTH,[ORDER DATE],GETDATE())
    AS MONTH_DIFF FROM ORDERS
```

SYNTAX: `DATEDIFF(datepart, startdate, enddate)`

💡 DATEDIFF counts boundary crossings, not full 24-hour periods — two dates a day apart but in different months already show a month-diff of 1.

COMMON USE CASES

Order age & fulfillment time

SLA and turnaround tracking

Customer tenure & cohort analysis

DATE FUNCTIONS · REAL-WORLD APPLICATION

10

 Year-to-Date (YTD) Filter

Combining DATEADD, GETDATE, YEAR, and CONCAT inside one WHERE clause — the exact nested-function pattern behind ‘this year so far’ dashboard filters.

```
--YTD (YEAR_TO_DATE) fetches start of current year till today's date
SELECT [ORDER DATE], SALES FROM ORDERS
WHERE [ORDER DATE] >= DATEADD(YEAR,-4,CONCAT(YEAR(GETDATE()), '-', '01-01'))
AND [ORDER DATE] <= DATEADD(YEAR,-4,GETDATE())
ORDER BY [ORDER DATE] ASC
```

WHY IT MATTERS:

This WHERE clause nests four functions — DATEADD, CONCAT, YEAR, and GETDATE — into one dynamic boundary that updates automatically, with no hardcoded dates. Because this Superstore data sits a few years in the past, the query shifts ‘today’ back by 4 years so the YTD window lands inside the data's real date range.

COMMON USE CASES

Rolling year-to-date offset

Dynamic — zero hardcoded dates

Powers live YTD sales dashboards

QUICK RECAP

DATE TOOLKIT

NOW() → CURRENT_TIMESTAMP · GETDATE · GETUTCDATE · SYSDATETIME

DATE PARTS → YEAR · MONTH · DAY

READ FORMAT → DATENAME · DATEPART

SHIFT DATES → DATEADD

BOUNDARIES → DATEFROMPARTS · EOMONTH

MEASURE GAP → DATEDIFF

REAL-WORLD → Year-to-Date (YTD) query

Following my Azure Data Engineering journey — one query at a time.

💡 Let's connect, and feel free to repost if this helped your revision too!